

Università degli Studi di Salerno

Corso di Ingegneria del Software

Easy Work Management System

Test Plan

Versione 0.5



E.W.M.S.

Easy Work Management System

Data: 20/12/2025

Progetto: Easy Work Management System	Versione: 0.5
Documento: Test plan	Data: 20/12/2025

Coordinatore del progetto:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Partecipanti:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Scritto da:	Vincenzo Mauro, Giovanni Robustelli
--------------------	-------------------------------------

Revision History

Data	Versione	Descrizione	Autore
19-12-2025	0.1	Strutturazione documento e inserimento punto 9	Giovanni Robustelli
20-12-2025	0.5	Sono stati aggiunti tutti i punti previsti, si prevede l'aggiunta di nuovi test cases	Giovanni Robustelli

Indice

1.	Introduzione	4
2.	Relazione con gli altri documenti	4
3.	System overview	4
4.	Funzionalità da testare/da non testare	5
5.	Pass/Fail criteria	6
6.	Approccio	7
7.	Sospensione e ripristino	8
8.	Materiale per i test	9
9.	Test cases	11
9.1	Test_Login (TC_1)	11
9.2	Test_Modifica_password (TC_2)	12
9.3	Test_Creazione_Task (TC_3)	13
9.4	Test_Sospensione_task (TC_4)	15
9.5	Test_Inserimento_account (TC_5)	16
10.	Testing Schedule	18

1. Introduzione

Questo documento ha l'obiettivo di definire la strategia di testing e le specifiche di validazione per la piattaforma Easy Work Management System (E.W.M.S.). In questo documento saranno presentate le strategie per il testing per il sistema. Saranno esplorate le tecniche che il team applicherà per effettuare unit testing, integration testing e system testing. In particolare, saranno elencati i test cases per le funzionalità critiche del sistema.

2. Relazione con gli altri documenti

Per la stesura di questo documento si è fatto riferimento ai seguenti documenti:

- **Requirement Analysis Document:** tale relazione si fonda sul fatto che i test case critici scelti si sono basati sui requisiti funzionale e non funzionali descritti all'interno del documento con particolare attenzione anche ai casi d'uso che prevedevano una forte interazione da parte degli attori del sistema.
- **System Design Document:** tale relazione si fonda sulla suddivisione in sottosistemi del sistema EWMS, grazie a questo documento è stato possibile individuare correttamente i sottosistemi che partecipano ai test case descritti in questo documento.
- **Object Design Document:** tale relazione è giustificata dal fatto che il documento contiene la descrizione delle classi che entreranno in gioco nei testing.

3. System overview

L'applicazione E.W.M.S. è una web application basata su una three-tier architecture, come descritto nel System Design Document. Questa struttura consente una netta separazione delle responsabilità, facilitando la decomposizione gerarchica necessaria per il testing modulare:

- **Presentation Layer:** Gestisce l'interazione con l'utente finale tramite interfacce web dinamiche. È il livello in cui vengono acquisiti i dati di input attraverso i form, visualizzati i feedback del sistema e gestito il flusso di controllo.
- **Application Layer:** Rappresenta il nucleo dell'applicazione dove risiede la business logic. In questo livello, i vari service elaborano le richieste e applicano i vincoli definiti nel RAD.

- Data Layer: Responsabile della persistence dei dati, interagisce con il database relazionale. Utilizza il pattern DAO (Data Access Object) per garantire l'integrità delle informazioni durante i processi di lettura e scrittura.

Il sistema punta a centralizzare l'assegnazione dei compiti e il monitoraggio dello stato delle attività, riducendo i tempi di coordinamento e aumentando l'efficienza e la trasparenza organizzativa.

4. Funzionalità da testare/da non testare

In questa sezione viene riportata l'applicazione sistematica del metodo category partition per le principali funzionalità del sistema E.W.M.S., identificate come critiche nel Requirement Analysis Document. L'obiettivo è trasformare i functional requirements in una serie di test frames che garantiscano una copertura esaustiva degli scenari di utilizzo, dei casi di error e dei boundary values. Le funzionalità che sono state scelte per avere un testing più approfondito sono:

- Login
- Modifica della password
- Creazione di un nuovo task
- La sospensione di un task
- Creazione di un nuovo account da aggiungere al sistema

Il sistema presenta diverse altre funzionalità, le quali però hanno un'interazione leggera con gli attori del sistema o hanno un funzionamento molto simile alle funzionalità testate, la loro fase di testing quindi si condurrà principalmente all'interno di unit e integration testing.

5. Pass/Fail criteria

In questa sezione vengono definiti i criteri oggettivi per determinare l'esito delle attività di testing condotte sul sistema E.W.M.S.. La valutazione si basa sulla coerenza tra il comportamento osservato dell'applicazione e le specifiche tecniche formalizzate nel Requirement Analysis Document e nell'Object Design Document.

Criteri di Superamento

Un **test case** è considerato superato quando il sistema soddisfa contemporaneamente le seguenti condizioni:

- **Integrità della Logica:** L'output prodotto dall'application layer corrisponde esattamente ai risultati attesi definiti nell'oracolo del caso di test.
- **Consistenza della Persistenza:** Le operazioni di CRUD effettuate tramite il pattern DAO si riflettono correttamente nel database. (Ad esempio, la creazione di un task deve generare un nuovo record e il relativo path dell'allegato se presente).
- **Feedback dell'Interfaccia:** La web interface mostra il messaggio di conferma o di errore previsto.

Criteri di Fallimento

Un **test case** è considerato fallito se si verifica anche solo una delle seguenti condizioni:

- **Deviazione dalle Specifiche:** Il sistema produce un output differente da quello previsto, o non attiva le post-condizioni definite nell'ODD.
- **Mancata Validazione:** Il sistema accetta dati di input che violano i vincoli di boundary values (es. accettazione di una password troppo corta o di una data di scadenza passata).
- **Inconsistenza dei Dati:** L'operazione non viene riportata correttamente nel persistence layer (es. dati mancanti o errati nel database nonostante un messaggio di successo).
- **Eccezioni non Gestite:** L'esecuzione del test solleva una exception non intercettata (es. NullPointerException o SQLException) che causa l'interruzione del processo o la visualizzazione di una pagina di errore generica non formattata.

Criteri per la qualità della fase di testing

La fase di testing è una delle parti fondamentali durante lo sviluppo di un sistema. Il suo svolgimento può anche coprire il 25% degli sforzi totali di sviluppo sia in senso di tempo che in senso di costo. Quindi la **qualità** della fase di testing viene misurata in base a quanti test hanno effettivamente provocato delle failure all'interno del sistema. Per un testing di successo si prevede che la maggior parte dei test case falliscano alla fine di individuare tutte i possibili difetti che inevitabilmente saranno presenti durante lo sviluppo del sistema.

6. Approccio

L'approccio adottato per la verifica del sistema E.W.M.S. segue una metodologia di black-box testing rigorosa integrata con fasi di white-box testing volte a dare copertura totale al testing delle singole parti del sistema.

Unit testing

L'obiettivo dello unit testing è verificare la correttezza logica dei singoli componenti del sistema in modo isolato. Per E.W.M.S., questo livello di testing si focalizza sulle classi definite nell'Object Design Document e si concentrerà sui servizi offerti da ogni singolo sottosistema.

- **Analisi dei contratti:** Verifica rigorosa delle pre-conditions e post-conditions specificate nell'ODD.
- **Boundary Value Analysis:** Test dei limiti per ogni parametro di input (es. lunghezze stringhe, valori nulli).
- **Mocking:** Utilizzo di mock objects o stubs per isolare l'unità sotto esame dalle dipendenze esterne, come il database o il web container, garantendo che l'errore rilevato appartenga esclusivamente all'unità testata.

Per verificare che l'intero codice sia testato in questa fase verranno utilizzate anche tecniche di white-box testing a discrezione dello sviluppatore.

Integration testing

L'integration testing ha l'obiettivo di verificare le interazioni tra i diversi moduli della three-tier architecture. Per il sistema E.W.M.S., si è deciso di adottare la tecnica del sandwich testing. Questo approccio permette di testare simultaneamente l'applicazione dall'alto verso il basso e dal basso verso l'alto, convergendo verso l'application layer identificato come strato target.

La decomposizione gerarchica del sistema per questa fase prevede la suddivisione in tre livelli logici:

- **Strato Superiore:** Composto dai controller del presentation layer che gestiscono le interfacce utente.
- **Strato Target:** Rappresentato dai componenti del application layer dove risiede la business logic.
- **Strato Inferiore:** Costituito dal persistence layer (**DAO**).

System testing

Il system testing rappresenta la fase finale di verifica, in cui l'intera applicazione viene testata come un'unica entità per accertarsi che soddisfi i functional requirements descritti nel Requirement Analysis Document.

L'Oggetti del test sarà l'intero processo software. La tecnica utilizzata sarà **category partition**: Applicazione sistematica delle tabelle di partizione generate (es. per il login o la creazione di un task) per coprire tutti gli scenari d'uso nominali e di error delle funzionalità critiche descritte all'interno di questo documento.

7. Sospensione e ripristino

In questa sezione vengono definiti i criteri secondo i quali le attività di testing sul sistema E.W.M.S. devono essere temporaneamente interrotte e le condizioni necessarie per la ripresa del processo di verifica. Questa gestione assicura che le risorse non vengano sprecate in presenza di bug bloccanti che impediscono la progressione dei test.

Criteri di Sospensione

L'attività di testing viene sospesa qualora si verifichi una delle seguenti condizioni critiche:

- **Bug bloccanti:** Presenza di malfunzionamenti critici nelle funzionalità core che impediscono l'accesso ad altre aree del sistema. (Ad esempio: impossibilità di inizializzare la sessione utente o errori fatali nel connection pool).
- **Elevato tasso di fallimento:** Se oltre il 40% dei test frames eseguiti in una specifica sessione produce un esito di fail, indicando un'instabilità diffusa della build corrente.
- **Corruzione dei dati:** Rilevamento di anomalie gravi nel persistence layer che compromettono l'integrità dei record durante i test di integrazione o di sistema.

7.2 Requisiti di Ripristino

Una volta sospesa l'attività, il **testing** può essere ripreso solo dopo il soddisfacimento dei seguenti requisiti:

- **Risoluzione delle anomalie:** Il team di sviluppo deve fornire una nuova build in cui il bug che ha causato la sospensione è stato corretto e verificato tramite uno unit testing preliminare.
- **Ripristino dell'ambiente:** Verifica della corretta configurazione del server di deployment e ripristino della consistenza del database (attraverso procedure di backup e recovery se necessario).
- **Disponibilità della documentazione:** Aggiornamento dell'Object Design Document o del System Design Document qualora la correzione del problema abbia comportato modifiche strutturali alle class interfaces o alla logica di service.

L'obiettivo di questa strategia è garantire che il modello del sistema mantenga standard qualitativi elevati, evitando che errori strutturali nello strato della business logic o incongruenze con il modello si propaghino nelle fasi avanzate di validazione.

8. Materiale per i test

Per l'esecuzione dei test sul sistema E.W.M.S., verranno adottati strumenti specifici del software engineering per l'ambiente Java, garantendo l'automazione dei processi e la ripetibilità dei test cases. La scelta degli strumenti è dettata dalla necessità di coprire i tre livelli di testing definiti nella metodologia, supportando la three-tier architecture del progetto.

Strumento per lo Unit Testing: JUnit

Per la verifica dei singoli componenti, verrà utilizzato **JUnit**.

- **Motivazione:** È il framework di testing standard per applicazioni Java. Permette di definire test cases atomici, gestire le pre-conditions tramite annotazioni come `@Before` e verificare i risultati attesi utilizzando le assertions.
- **Utilizzo:** Verrà impiegato per testare la validazione degli input nei service e la correttezza dei metodi dei DAO, assicurando che ogni unità rispetti le specifiche dell'ODD.

Strumento per l'Integration Testing: Mockito e DBUnit

Per gestire la complessità del **sandwich testing** e l'integrazione tra i livelli, si consiglia l'accoppiata di questi due strumenti:

- **Mockito:** È fondamentale per la creazione di **mock objects** e **stubs**. Nel nostro approccio, Mockito permetterà di isolare lo strato target simulando le risposte del persistence layer senza dover interrogare il database reale, o simulando le chiamate provenienti dal presentation layer.
- **DBUnit:** Estensione di JUnit, è ideale per l'integrazione con il database. Consente di mettere il database in uno stato noto prima di ogni test (tramite datasets in formato XML), verificando che le operazioni di CRUD eseguite dai DAO producano la persistenza corretta dei dati.

Strumento per il System Testing: Selenium

Per la validazione end-to-end dell'applicazione, verrà utilizzato Selenium WebDriver.

- **Motivazione:** Permette di automatizzare le azioni degli utenti sui browser web. È ideale per verificare i flussi descritti nel RAD, interagendo direttamente con il presentation layer.
- **Utilizzo:** Verrà impiegato per simulare scenari completi, come il login, la creazione di un task (compreso l'upload di allegati). Selenium controllerà che, a fronte di input validi o errati, il DOM della pagina mostri correttamente i messaggi di successo o i messaggi di error definiti nei requisiti.

Ambiente di Esecuzione

Tutti i test saranno integrati all'interno dell'IDE e potranno essere eseguiti tramite strumenti di build automation come Maven.

9. Test cases

In questa sezione verranno inserite le tabelle del Category Partition per le funzionalità selezionate per il testing, ogni test case avrà un Test case id (TC_id) e un nome utilizzati per il riferimento nel documento di Test Case Specification.

9.1 Test_Login (TC_1)

Category	Partizione	Vincoli
Username (email)	Valido (presente nel db)	[property Valido]
	Inesistente	[error]
	Formato non valido	[error]
	Null o vuoto	[error]
Password	Corretta (corrispondente all'username)	[if Valido]
	Errata	[if Valido] [error]
	Null o vuota	[error]

Tabella formati

Campo	Formato
Username (email)	^[a-zA-Z0-9]{1,50}\.[a-zA-Z0-9]{1,50}@azienda\.it\$
Password	^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\$!%*?&])[A-Za-z\d@\$!%*?&]{8,16}\$

Test frames

ID Frame	Username	Password	Risultato Atteso (Output)
TF_1.1	Valido	Corretta	Login Successo: Redirect a Homepage
TF_1.2	Valido	Errata	Error: "Username o password non corrette"
TF_1.3	Inesistente	Qualsiasi	Error: "Username o password non corrette"
TF_1.4	Null	Qualsiasi	Error: "Il campo e-mail è obbligatorio"
TF_1.5	Valido	Null	Error: "Il campo password è obbligatorio"
TF_1.6	Formato Errato	Qualsiasi	Error: "Il formato e-mail non valido"

9.2 Test_Modifica_password (TC_2)

Category	Partizione	Vincoli
Password attuale	Corretta (coincide con il DB)	[property Valido]
	Errata	[error]
	Null o vuota	[error]
Nuova password	Valida	[property nuovaValido]
	Formato non valido	[error]
	Null o vuota	[error]
Conferma Password	Identica a “Nuova Password”	[if nuovaValida]
	Differente dalla “Nuova Password”	[error]

Tabella dei formati

Campo	Formato
Password	^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\$!%*?&])[A-Za-z\d@\$!%*?&]{8,16}\$

Test frames

ID Frame	Password Attuale	Nuova Password	Conferma Password	Risultato Atteso (Output)
TF_2.1	Corretta	Valida	Identica	Success: Password aggiornata e redirect
TF_2.2	Errata	Valida	Identica	Error: "Password attuale errata"
TF_2.3	Corretta	Valida	Differente	Error: "Le nuova password inserite non coincidono"
TF_2.4	Corretta	Formato non valido	N/A	Error: "La nuova password non è sufficientemente sicura"
TF_2.5	Vuota	Valida	Identica	Error: "Il campo 'password attuale' è obbligatorio"

9.3 Test_Creazione_Task (TC_3)

Category	Partizione	Vincoli
Titolo	Valido	[property TitoloOK]
	Vuoto	[error]
	Formato non valido	[error]
Dipendente	Selezionato (presente in lista)	[property DipSelected]
	Non selezionato	[error]
Data di scadenza	Futura	[property DataOK]
	Passata	[error]
	Vuota	[error]
Priorità	Selezionata	[property PrioritàOK]
	Non selezionata	[error]
Istruzioni	Valida	[property IstruzioniOK]
	Formato non valido	[error]
Allegato	Valido	[property FileOK]

	Formato non supportato	[error]
	Dimensione eccessiva	[error]
	Assente	

Tabella formati

Campo	Formato
Titolo	^[a-zA-Z0-9\s.,\-_!]{1,50}\$
Data di scadenza	^\d{4}-(0[1-9] 1[0-2])-(0[1-9] [12][0-9] 3[01])\$ (formato YYYY-MM-DD)
Istruzioni	^[\\s\\S]{10,2000}\$
Allegato	.pdf, .docx, .xlsx, dimensione < 10 MB

Test frames

ID Frame	Titolo	Dipendente	Data	Istruzioni	Allegato	Risultato Atteso (Output)
TF_3.1	Valido	Selezionato	Futura	Valida	Assente	Success: Task creato, Redirect a Homepage
TF_3.2	Valido	Selezionato	Futura	Valida	Valido	Success: Task e attachment salvati
TF_3.3	Vuoto	Selezionato	Futura	Valida	N/A	Error: "Il campo 'Titolo' è obbligatorio"
TF_3.4	Valido	Non selezionato	Futura	Valida	N/A	Error: "Per favore scegliere un dipendente"
TF_3.5	Valido	Selezionato	Errata	Valida	N/A	Error: "Data di scadenza obbligatoria e in formato valido"
TF_3.6	Valido	Selezionato	Futura	Formato non valido	N/A	Error: "Inserire istruzioni (min. 10, max 2000 caratteri)"

TF_3.7	Valido	Selezionato	Futura	Valida	Formato non supportato	Error: "Inserito file non consentito"
TF_3.8	Valido	Selezionato	Futura	Valida	Dimensione eccessiva	Error: "Il file supera la dimensione massima"

9.4 Test_Sospensione_task (TC_4)

Category	Partizione	Vincoli
Stato iniziale task	In Elaborazione	[property StatoValido]
	Da completare	[error]
	Sospeso	[error]
	Completato	[error]
Motivazione	Valida	[property MotivazioneOK]
	Formato non valido	[error]
	Null o vuota	[error]

Tabella formati

Campo	Formato
Motivazione	^[s\S]{10,500}\$

Test frames

ID Frame	Stato Iniziale	Motivazione	Risultato Atteso (Output)
TF_4.1	In elaborazione	Valida	Success: Stato diventa Sospeso , invio notifica

TF_4.2	In elaborazione	Formato non valido	Error: "È obbligatorio inserire una motivazione... (minimo 10, max 500 caratteri)"
TF_4.3	Da completare	N/A	Error: possibilità di sospendere il task non disponibile
TF_4.4	Completato	N/A	Error: possibilità di sospendere il task non disponibile
TF_4.5	Sospeso	N/A	Error: possibilità di sospendere il task non disponibile
TF_4.6	In elaborazione	Null	Error: Motivazione obbligatoria

9.5 Test_Inserimento_account (TC_5)

Category	Partizione	Vincoli
Nome	Valido	[property NomeOK]
	Vuoto	[error]
	Formato non valido	[error]
Cognome	Valido	[property CognomeOK]
	Vuoto	[error]
	Formato non valido	[error]
Email	Valido	[property EmailOK]
	Formato non valido	[error]
	Già presente nel DB	[error]
	Vuoto	[error]
Data di nascita	Valido	[property DataOK]
	Formato non valido	[error]
	Vuoto	[error]
Ruolo	Selezionato	
	Non selezionato	[error]

Matricola	Generata dal sistema	[property MatGenOK]
Password	Generata	[property passgenOK]
	Non generata	[error]

Tabella formati

Campo	Formato
Nome	^[a-zA-Z]{1,50}\$
Cognome	^[a-zA-Z]{1,50}\$
Email	^[a-zA-Z0-9]{1,50}\.[a-zA-Z0-9]{1,50}@azienda\.it\$
Data di nascita	^\d{4}-(0[1-9] 1[0-2])-(0[1-9] 12)[0-9]{3}[01]\$ (formato YYYY-MM-DD)
Password	^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\$!%*?&])[A-Za-z\d@\$!%*?&]{8,16}\$

Test frames

ID Frame	Nome	Cognome	Data di Nascita	E-mail	Password	Risultato Atteso (Output)
TF_5.1	Valido	Valido	Valido	Valida (nuova)	Generata	Success: Account creato e salvato
TF_5.2	Valido	Valido	Valido	Già esistente	Generata	Error: "Indirizzo e-mail già utilizzato"
TF_5.3	Vuoto	Valido	Valido	Valida	Generata	Error: "Il campo 'nome' è obbligatorio"
TF_5.4	Valido	Valido	Formato non valido	Valida	Generata	Error: "Formato data non valido"
TF_5.5	Valido	Valido	Vuoto	Valida	Generata	Error: "Il campo 'data di nascita' è obbligatorio"
TF_5.6	Valido	Valido	Valido	Formato errato	Generata	Error: "Il formato e-mail non è valido"

TF_5.7	Valido	Valido	Valido	Valida	Vuota	Error: "Password obbligatoria"
TF_5.8	Formato non valido	Valido	Valido	Valida	Generata	Error: "Nome troppo lungo (Max 50 caratteri)"
TF_5.9	Valido	Vuoto	Valido	Valida	Generata	Error: "Campo 'cognome' obbligatorio"

10. Testing Schedule

Le attività di testing per il sistema E.W.M.S. non sono concepite come una fase isolata al termine dello sviluppo, ma sono integrate nell'intero ciclo di vita del software engineering. Il processo prevede che lo unit testing venga eseguito non appena le singole unità logiche risultano pronte e compilabili. Questo approccio incrementale consente l'individuazione immediata di eventuali regressioni o difetti strutturali, riducendo i costi di correzione nelle fasi avanzate di integrazione.

Inoltre, la suite di test è considerata un'entità dinamica. Durante lo sviluppo, il team aggiungerà nuovi test cases man mano che lo sviluppo porta ad un miglior comprensione del sistema e dei suoi requisiti o qualora emergano scenari d'uso non previsti inizialmente o nel caso in cui la scomposizione gerarchica delle nuove funzionalità riveli ulteriori boundary values critici. Questo garantisce che la copertura dei functional requirements rimanga completa e aggiornata rispetto all'evoluzione della business logic e delle interfacce definite nell'Object Design Document.